

FastAPI 기반 LLM 백엔드 API 개발과 스트리밍 실습

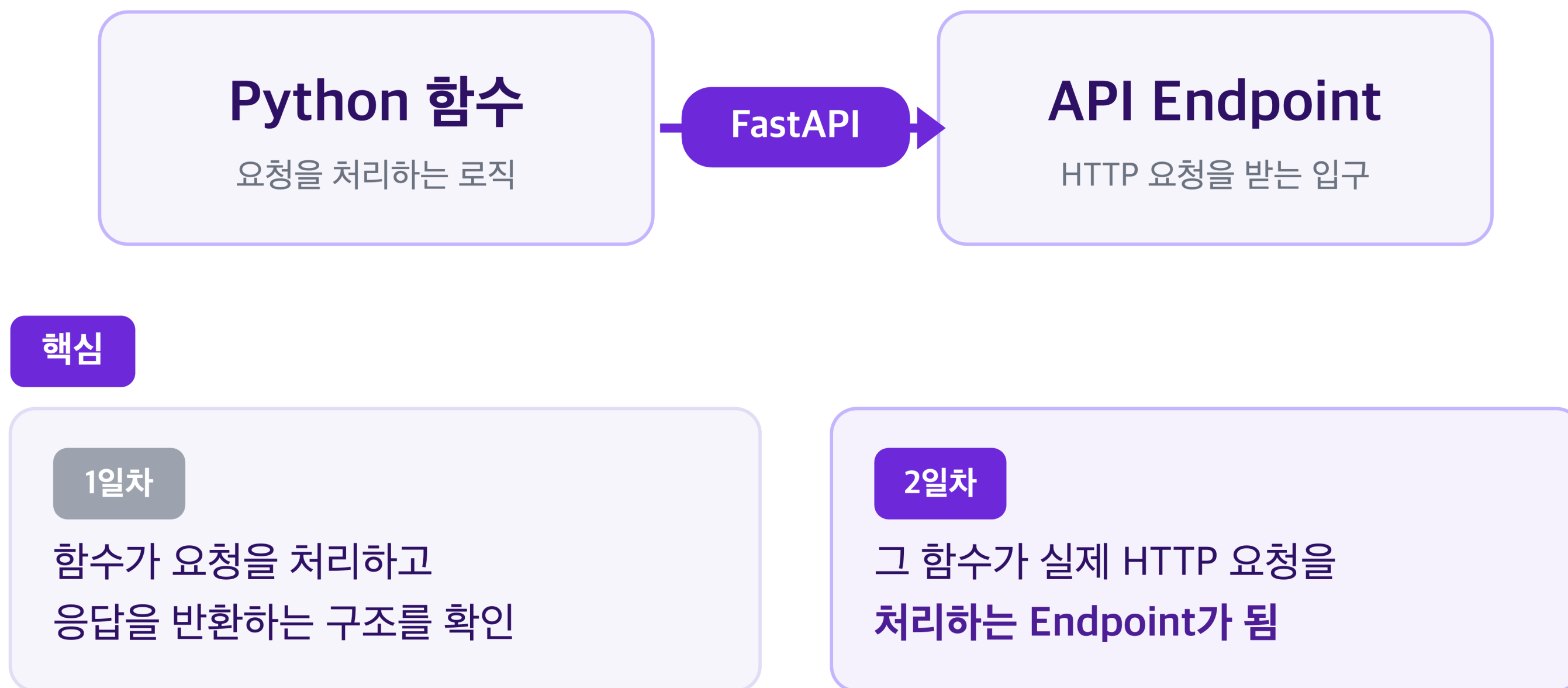
02 FastAPI로 백엔드 API 만들기

FastAPI 구조 이해 및 서버 실행



FastAPI 란?

- FastAPI는 Python 함수로 API Endpoint를 만들 수 있게 해주는 웹 프레임워크



FastAPI 서버의 기본 구성

구성 요소	역할
main.py	서버 코드가 작성되는 기본 파일
FastAPI()	API 앱 객체 생성
@app.get()	특정 URL 요청을 함수와 연결
Endpoint 함수	요청을 처리하고 응답 반환
Uvicorn	FastAPI 앱을 실행하는 서버



기본 파일 구조

- 처음에는 복잡한 프로젝트 구조보다 단순한 구조로 시작
- 아직 파일을 폴더로 분리하지 않고, `main.py` 한 파일로 시작

● ● ● 프로젝트 구조

day2_fastapi/

└─ main.py

main.py 에 들어가는 것

FastAPI import

app 객체 생성

기본 Endpoint 함수 작성



FastAPI 앱 객체 만들기

- app은 FastAPI 서버의 **중심 객체**
 - Endpoint를 등록하는 대상
 - 요청을 받을 API 앱
 - Uvicorn이 실행할 객체
- main.py 안의 app 객체를 uvicorn이 실행

● ● ● main.py

```
from fastapi import FastAPI  
  
app = FastAPI()
```



기본 Endpoint 작성하기

- 코드 의미:

- GET / 요청이 들어오면
- read_root() 함수가 실행되고
- 반환값이 JSON 응답으로 전달

- 확인할 것:

- 함수의 반환값이 dict 형태인가?
- 브라우저에서 JSON 응답으로 보이는가?

main.py

```
@app.get("/")
def read_root():
    return {
        "message": "Hello FastAPI"
    }
```



전체 코드 확인

- 가장 작은 FastAPI 서버 예시
- FastAPI 앱 생성 -> Endpoint 등록 -> 요청 처리 함수 작성 -> 응답 반환

main.py

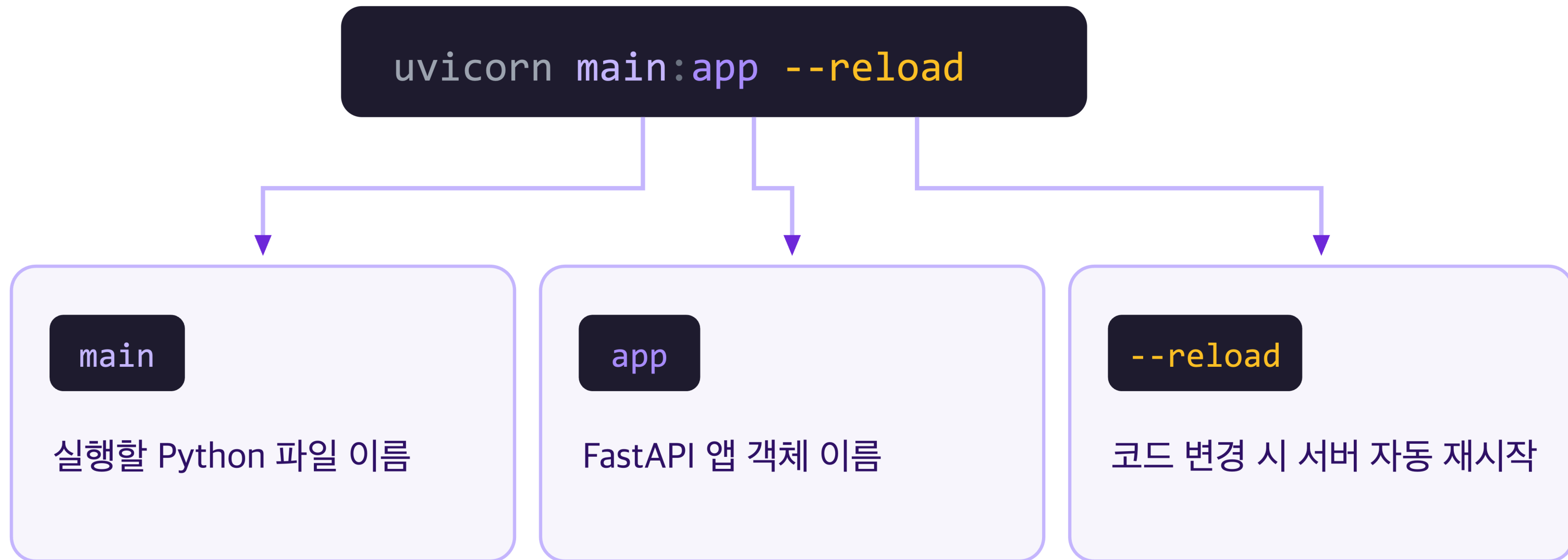
```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {
        "message": "Hello FastAPI"
    }
```



Uvicorn으로 서버 실행하기



- 실행 후 서버 주소가 출력됨

```
http://127.0.0.1:8000
```



브라우저에서 응답 확인하기

- 브라우저에서 다음 주소로 접속

● ● ● 브라우저 응답

브라우저로 접속

`http://127.0.0.1:8000`

돌아오는 응답 (JSON)

```
{ "message": "Hello FastAPI" }
```

- 확인할 것:
 - 서버가 정상 실행되었는가?
 - 브라우저에서 JSON 응답이 보이는가?
 - Python dict 반환값이 JSON 응답으로 변환되었는가?



서버 실행 중 자주 보는 메시지

- Uvicorn 실행 시 터미널에는 서버 상태가 출력됨

● ● ● 터미널

서버를 실행하면

Uvicorn running on http://127.0.0.1:8000

요청이 들어올 때마다

GET / 200 OK

- 확인할 것:
 - 어떤 URL로 요청이 들어왔는가?
 - 어떤 Status Code가 반환되었는가?
 - 서버가 종료되지 않고 실행 중인가?



코드 수정 후 다시 확인하기

- message 값을 바꾸고 진행

```
main.py

@app.get("/")
def read_root():
    return {
        "message": "FastAPI 서버가 살아 있습니다."
    }
```

- --reload 옵션을 사용하면 코드 변경 후 서버가 자동으로 다시 로드됨
- 확인할 것:
 - 코드를 수정했을 때 응답이 바뀌는가?
 - 서버를 매번 수동으로 껐다 켜지 않아도 되는가?



환경변수 파일의 역할

- 프로젝트에서는 설정값을 코드에 직접 쓰지 않고 별도 파일로 관리하는 경우가 많음
- .env 파일에는 다음과 같은 값을 둘 수 있음

● ● ● .env

```
OPENAI_API_KEY=sk-xxxxx
```

```
APP_PORT=8000
```

- 설정값을 별도 파일로 관리할 수 있다는 정도만 이해



확인 체크리스트

- ✓ `main.py` 파일을 만들 수 있다
- ✓ `FastAPI()` 로 app 객체를 만들 수 있다
- ✓ `@app.get("/")` 로 기본 Endpoint를 작성할 수 있다
- ✓ `uvicorn main:app --reload` 로 서버를 실행할 수 있다
- ✓ 브라우저에서 JSON 응답을 확인할 수 있다
- ✓ `.env` 파일이 설정값을 관리하는 용도임을 이해한다

기본 Endpoint 구현과 요청 파라미터 적용

Endpoint는 요청의 입구

- Endpoint는 특정 HTTP Method와 URL Path에 연결된 처리 함수

```
main.py

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }
```

GET /health

Method + Path



health_check()

처리 함수 실행



JSON 응답

함수 반환값

Endpoint = 특정 HTTP Method와 URL Path에 연결된 처리 함수

GET Endpoint 만들기

- GET Endpoint는 데이터를 조회하거나 상태를 확인할 때 자주 사용

```
● ● ● main.py

@app.get("/items")
def get_items():
    return {
        "items": ["book", "pen", "notebook"]
    }

# 응답 예시
{ "items": ["book", "pen", "notebook"] }
```

- 확인할 것:
 - 리스트 데이터도 JSON 응답으로 반환되는가?
 - GET /items 요청과 함수가 연결되는가?

POST Endpoint 만들기

- POST Endpoint는 서버로 데이터를 보내거나 처리 요청을 할 때 사용

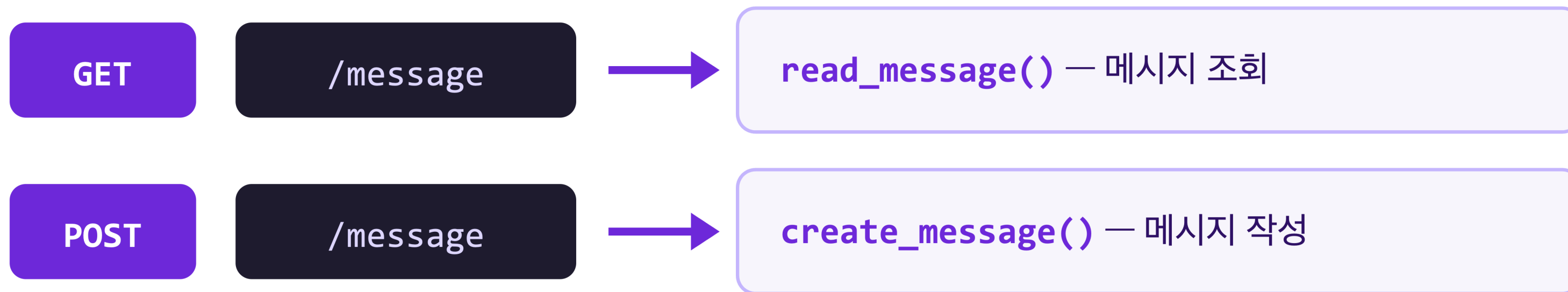
```
main.py

@app.post("/echo")
def echo_message():
    return {
        "message": "POST 요청을 받았습니다."
    }
```

- 확인할 것:
 - 같은 서버 안에 GET과 POST Endpoint를 함께 만들 수 있는가?
 - Method가 달라지면 다른 함수가 실행되는가?

같은 Path, 다른 Method

- 같은 Path라도 Method가 다르면 다른 Endpoint로 처리할 수 있음



같은 Path라도 Method가 다르면 FastAPI가 다른 함수로 연결합니다

- 확인할 것:
 - GET /message와 POST /message가 다른 함수로 연결되는가?
 - FastAPI는 Method와 Path 조합으로 Endpoint를 구분하는가?

Path Parameter란?

- Path Parameter는 URL 경로 안에 포함되는 값
- FastAPI에서는 중괄호 {}로 표현

main.py

```
# 요청 - GET /items/3
@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {
        "item_id": item_id
    }
```

- 확인할 것:
 - URL의 일부가 함수 인자로 전달되는가?
 - item_id가 숫자값으로 처리되는가?

Path Parameter 실행 예시

- Path Parameter는 특정 대상을 식별할 때 자주 사용

● ● ● 요청 / 응답

요청

GET /items/3

응답

{ "item_id": 3 }

자주 쓰는 형태

/users/1

/posts/10

/orders/2026

Path Parameter 타입 지정

- FastAPI에서는 함수 인자에 타입 힌트를 줄 수 있음

```
main.py

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {
        "user_id": user_id
    }

# /users/1    → 정상
# /users/abc → 타입 오류
```

- 확인할 것:
 - 숫자 형태의 URL 값이 int로 전달되는가?
 - 숫자가 아닌 값이 들어오면 어떻게 되는가?

Query Parameter란?

- Query Parameter는 URL 뒤에 붙는 조회 조건
- FastAPI에서는 함수 인자로 받을 수 있음

main.py

```
# 요청 - /search?keyword=fastapi
@app.get("/search")
def search_items(keyword: str):
    return {
        "keyword": keyword
    }
```

- 확인할 것:
 - URL 뒤의 값이 함수 인자로 전달되는가?
 - keyword 값이 응답에 포함되는가?

Query Parameter 기본값

- Query Parameter에는 기본값을 줄 수 있음

● ● ● main.py

```
@app.get("/search")
def search_items(keyword: str = "all"):
    return {
        "keyword": keyword
    }

# /search                → keyword = all
# /search?keyword=fastapi → keyword = fastapi
```

- 확인할 것:
 - Query Parameter가 없을 때 기본값이 사용되는가?
 - 값을 전달하면 기본값이 바뀌는가?

여러 Query Parameter 받기

- Query Parameter는 여러 개를 함께 받을 수 있음

```
● ● ● main.py

@app.get("/search")
def search_items(keyword: str = "all", limit: int = 10):
    return {
        "keyword": keyword,
        "limit": limit
    }

# /search?keyword=fastapi&limit=5
# → { "keyword": "fastapi", "limit": 5 }
```

- 확인할 것:
 - 여러 Query Parameter가 각각 함수 인자로 전달되는가?
 - limit 값이 숫자로 처리되는가?

Path와 Query 함께 사용하기

- Path와 Query Parameter는 함께 사용할 수 있음
- user_id는 경로에서 대상 지정
- limit은 Query로 조회 개수 조절

● ● ● main.py

```
@app.get("/users/{user_id}/posts")
def get_user_posts(user_id: int, limit: int = 10):
    return {
        "user_id": user_id,
        "limit": limit
    }

# GET /users/1/posts?limit=5
# → { "user_id": 1, "limit": 5 }
```

Path와 Query의 역할 구분

- Path Parameter와 Query Parameter는 모두 URL을 통해 값을 전달하지만 **역할이 다름**

구분	주로 쓰는 경우	예시
Path Parameter	특정 대상 식별	/users/1
Query Parameter	조회 조건 조절	?limit=5

Path Parameter는 '대상'을 고르고, Query Parameter는 '조건'을 조절합니다

간단한 예제 API 구성

- 확인할 것:
 - 여러 Endpoint를 하나의 FastAPI 앱에 등록할 수 있는가?
 - Method와 Path가 각각 다른 함수와 연결되는가?

● ● ● main.py

```
@app.get("/health")
def health_check():
    return {"status": "ok"}

@app.get("/items")
def get_items():
    return {"items": ["book", "pen", "notebook"]}

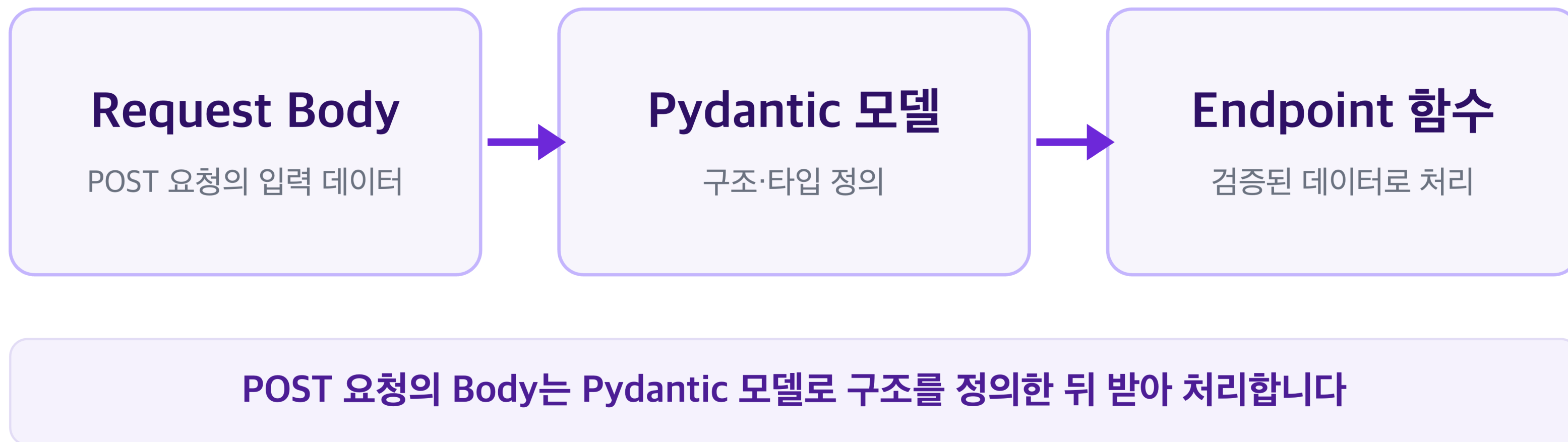
@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {"item_id": item_id}

@app.get("/search")
def search_items(keyword: str = "all", limit: int = 10):
    return {"keyword": keyword, "limit": limit}
```

Request Body와 Pydantic 모델 이해

Body 데이터 처리

- POST 요청에서 Body는 서버가 처리할 실제 입력 데이터
- FastAPI에서는 이 Body 데이터를 함수 인자로 직접 받기보다, 보통 **모델로 구조를 정의한 뒤 받아 처리함**



Pydantic이란?

- Pydantic은 Python에서 데이터의 구조와 타입을 정의하고 검증할 수 있게 해주는 라이브러리
- FastAPI에서는 요청 Body를 받을 때 Pydantic 모델을 자주 사용

models.py

```
from pydantic import BaseModel  
  
class MessageRequest(BaseModel):  
    message: str
```

- 위 코드는 다음 의미를 가짐
 - message라는 필드가 필요함
 - message는 문자열이어야 함
- Pydantic을 사용하면 요청 데이터가 정해진 구조에 맞는지 확인할 수 있음

Pydantic 모델이 필요한 이유

- 요청 Body는 항상 올바른 형태로 들어온다고 보장할 수 없음

● ● ● 입력 예시

```
# 요청 Body 는 항상 올바르게 보장할 수 없음
{} # message 누락
{ "message": 123 } # 타입 불일치 (숫자)
# Pydantic 모델이 입력 구조를 미리 정해 검증
```

- Pydantic 모델을 사용하면 요청 데이터의 구조를 미리 정할 수 있음
 - 필요한 필드 정의
 - 필드 타입 정의
 - 기본값 설정
 - 잘못된 입력을 검증할 기준 제공
- 핵심: Pydantic 모델은 “이 API가 어떤 입력을 기대하는지”를 정의한다.

Request Model이란?

- Request Model은 클라이언트가 보내는 입력 Body의 구조를 정의함

models.py

```
from pydantic import BaseModel

class MessageRequest(BaseModel):
    message: str

# 요청 예시
{ "message": "hello fastapi" }
```

- 확인할 것:
 - 어떤 필드를 받을지 정의함
 - 각 필드의 타입을 지정함
 - 요청 Body 구조가 코드에 명확히 드러남

Request Body 받기

```
● ● ● main.py

@app.post("/message")
def create_message(request: MessageRequest):
    return {
        "message": request.message
    }
```

- 확인할 것:
 - 요청 Body가 MessageRequest 모델로 들어옴
 - request.message로 입력값에 접근함
 - 반환값은 JSON 응답으로 전달됨

여러 필드와 기본값

- 요청 Body에는 여러 값을 함께 담을 수 있음

```
models.py

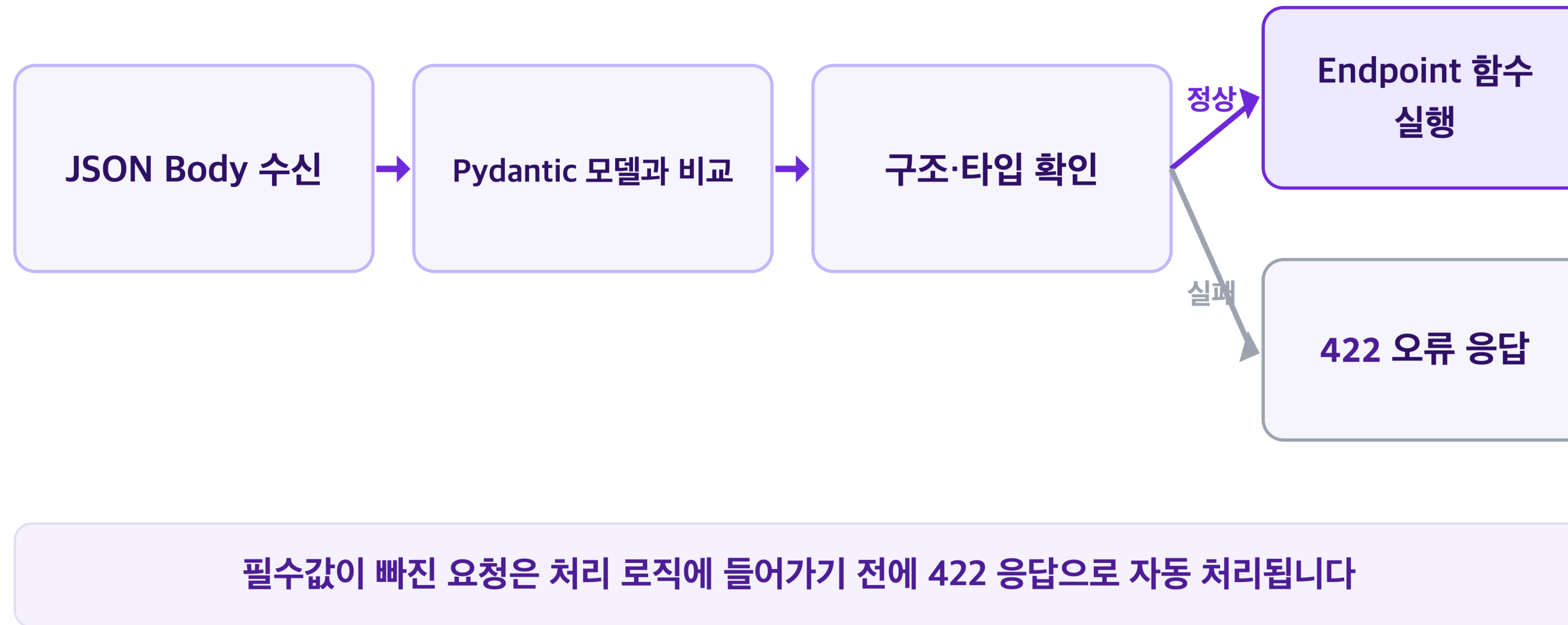
class TextRequest(BaseModel):
    text: str
    mode: str = "upper"

# 입력 예시
{ "text": "hello fastapi", "mode": "upper" }
```

- 확인할 것:
 - text는 필수 입력값
 - mode는 기본값이 있는 선택 입력값
 - 요청 구조가 코드에서 명확해짐

Validation 흐름 확인

- FastAPI는 요청 Body를 Endpoint 함수에 전달하기 전에 Pydantic 모델 기준으로 검증



- 예시)
 - 정상 요청이면 TextRequest모델로 변환되어 함수에 전달
 - 필수값이 빠진 요청은 처리 로직으로 들어가기 전에 **422 응답으로 자동 처리**

Request Model을 사용한 처리 흐름

- 흐름:

● ● ● main.py

```
@app.post("/text")
def process_text(request: TextRequest):
    if request.mode == "upper":
        result = request.text.upper()
    else:
        result = request.text
    return {
        "result": result
    }
```

흐름: Body 수신 → TextRequest 변환 → text/mode 사용 → 결과 반환

Response Model이란?

- Response Model은 서버가 반환하는 응답 Body의 구조를 정의함

models.py

```
class TextResponse(BaseModel):  
    result: str  
  
# 응답 예시  
{ "result": "HELLO FASTAPI" }
```

- Response Model을 사용하면 API가 어떤 응답을 반환하는지 코드에서 명확히 표현할 수 있다

Response Model 적용하기

```
main.py

@app.post("/text", response_model=TextResponse)
def process_text(request: TextRequest):
    result = request.text.upper()
    return {
        "result": result
    }
```

- 확인할 것:
 - 요청은 TextRequest 모델로 받음
 - 응답은 TextResponse 모델 구조에 맞춰 변환함
 - 입력과 출력의 구조가 코드에서 분리되어 보임

Request Model과 Response Model 비교

구분	역할	예시
Request Model	클라이언트가 보내는 입력 데이터 구조	TextRequest
Response Model	서버가 반환하는 응답 데이터 구조	TextResponse

Request Model은 '들어오는' 데이터 구조를, Response Model은 '나가는' 데이터 구조를 정의합니다

- 핵심:
 - Request Model은 들어오는 데이터의 구조를 정의
 - Response Model은 나가는 데이터의 구조를 정의

예제 코드 정리

- 확인할 것:
 - Request Model이 정의되어 있는가?
 - Response Model이 정의되어 있는가?
 - Endpoint 함수가 Request Model을 인자로 받는가?
 - 반환값이 Response Model 구조와 맞는가?

```
main.py

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TextRequest(BaseModel):
    text: str
    mode: str = "upper"

class TextResponse(BaseModel):
    result: str

@app.post("/text", response_model=TextResponse)
def process_text(request: TextRequest):
    result = request.text.upper()
    return {
        "result": result
    }
```

모델 사용 시 주의점

- Pydantic 모델을 사용할 때는 **필드 이름과 타입을 명확히** 작성해야 함

1 Key 이름 일치

요청 JSON의 key와 모델 필드명이 같아야 함

2 필수값과 기본값 구분

어떤 필드가 필수인지 명확히 구분해야 함

3 타입 힌트 = 검증 기준

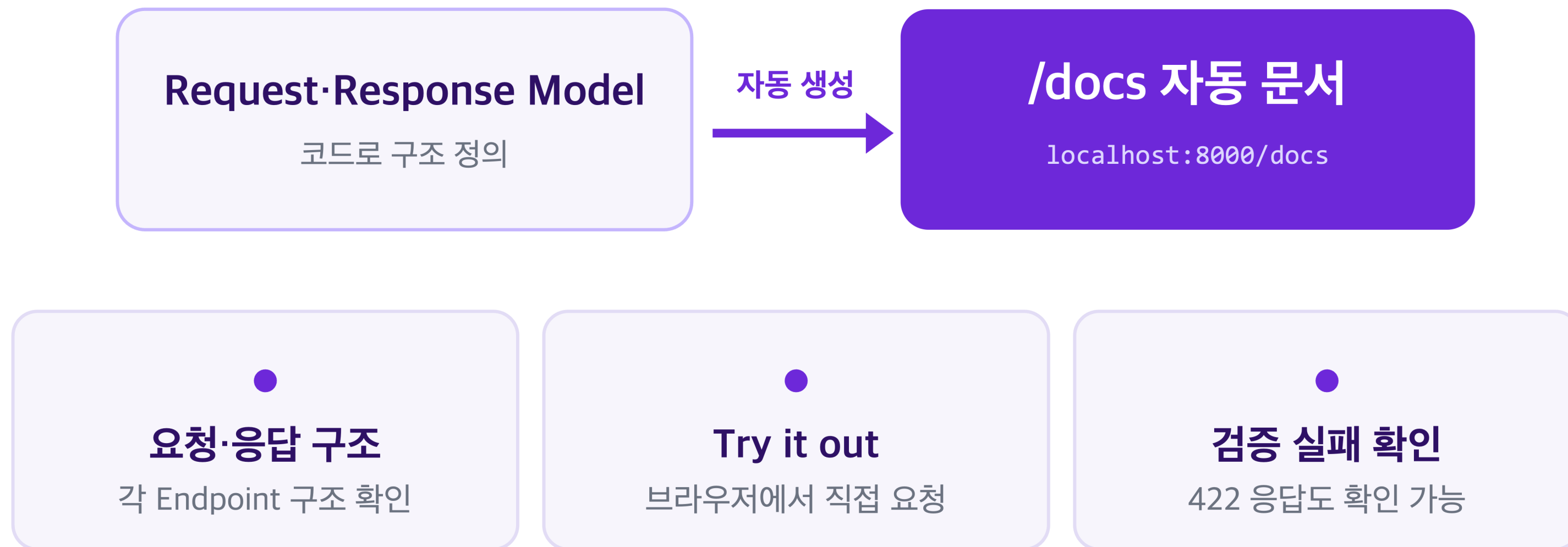
지정한 타입 힌트가 그대로 검증 기준이 됨

4 속성으로 값 접근

처리 로직에선 모델 객체의 속성으로 값에 접근

FastAPI 자동 문서 (Swagger, /docs)

- FastAPI는 정의된 Request / Response Model을 기반으로 API 문서를 자동 생성
- 서버 실행 후 브라우저에서 다음 주소로 접속
 - `http://localhost:8000/docs`



- 핵심: 별도 문서를 작성하지 않아도 **코드(Request / Response Model)**가 바로 문서가 됨

API 문서 확인 및 테스트 (실습)



실습 목표

- FastAPI 자동 문서와 TestClient를 활용해 Endpoint 구조를 확인하고, 정상 요청과 실패 요청의 응답 비교

실습 1. API 문서 확인 및 테스트

실습 목표

이번 실습에서는 FastAPI가 자동으로 제공하는 API 문서와 테스트 흐름을 확인합니다.

단순히 `/docs` 화면을 보는 것에서 끝내지 않고, 노트북 안에서 직접 요청을 보내며 다음 내용을 확인합니다.

- FastAPI 앱과 Endpoint 구성
- 자동 생성되는 API 문서 정보 확인
- Path Parameter 요청 테스트
- Query Parameter 요청 테스트
- Request Body 요청 테스트
- 정상 요청과 검증 실패 요청 비교



실습 흐름

1 라이브러리 불러오기

2 실습용 FastAPI 앱 만들기

3 TestClient로 요청 보내기

4 응답 출력 함수 만들기

5 /health 요청 테스트하기

6 Path Parameter 테스트하기

7 Query Parameter 테스트하기

8 Request Body 테스트하기

9 API 문서 정보 확인하기

10 서버 파일 저장 후 /docs 확인

TODO

정상·실패 요청 비교, OpenAPI Path 목록 확인 등을 직접 수행합니다



1. 라이브러리 불러오기

- FastAPI 앱 생성, 요청 테스트, Request Body 모델 정의에 필요한 라이브러리 호출

1. 라이브러리 불러오기

실습에 필요한 라이브러리를 불러옵니다.

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel
from pprint import pprint
import json
```

Python



2. 실습용 FastAPI 앱 만들기

- 테스트용 Endpoint 먼저 구성
- 실습 포인트:
 - GET·POST·Path·Query·Body 네 가지 입력 방식을 한 앱에 모아 두기
 - 다음 단계부터 같은 client로 차례로 두드리며 비교
 - 한 앱에 여러 Endpoint가 등록되는 구조 직접 확인

2. 실습용 FastAPI 앱 만들기

`/docs` 와 요청 테스트에 사용할 FastAPI 앱을 만듭니다.

이번 실습에서는 다음 Endpoint를 사용합니다.

```
GET    /health
GET    /items/{item_id}
GET    /search
POST   /message
```



2. 실습용 FastAPI 앱 만들기

- 테스트용 Endpoint 코드
- 실습 포인트:
 - @app.get + 함수 한 쌍이 Endpoint 하나 — **데코레이터가 매핑을 알아서 처리**
 - /items/{item_id}는 Path Parameter, /search는 Query Parameter — 함수 인자 비교

```
app = FastAPI()

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }

@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {
        "item_id": item_id
    }

@app.get("/search")
def search_items(keyword: str = "all", limit: int = 10):
    return {
        "keyword": keyword,
        "limit": limit
    }
```



2. 실습용 FastAPI 앱 만들기

- Request Body Endpoint 코드
- 실습 포인트:
 - MessageRequest 모델은 함수 위에 따로 — 한 번 만들면 여러 Endpoint에서 재사용
 - message 필드는 기본값 없음 = 필수값
 - 다음 단계에서 message 빠진 요청이 422로 자동 차단되는 모습 확인

```
@app.get("/search")
def search_items(keyword: str = "all", limit: int = 10):
    return {
        "keyword": keyword,
        "limit": limit
    }

class MessageRequest(BaseModel):
    message: str

@app.post("/message")
def create_message(request: MessageRequest):
    return {
        "message": request.message
    }
```



3. TestClient로 요청 보내기

- 노트북 안에서 우리 FastAPI 앱에 요청을 보내는 다리 — 서버 안 띄워도 호출 가능
- 사용 방식은 requests와 거의 동일 — `client.get()`, `client.post()`
- 이후 단계는 모두 '요청 → Status Code 확인 → Body 확인 → 정상·실패 비교' 반복

3. TestClient로 요청 보내기

브라우저의 `/docs` 화면을 열기 전에도, `TestClient` 를 사용하면 노트북 안에서 API 요청을 직접 보내볼 수 있습니다.

이렇게 하면 서버 요청과 응답을 코드로 바로 확인할 수 있습니다.

```
client = TestClient(app)
```

Python



4. 응답을 보기 좋게 출력하는 함수 만들기

- 요청 결과를 **같은 형식으로 정리**해 정상·실패를 셀 단위로 비교
- 실습 포인트:
 - 1일차 응답 요약 함수와 같은 발상 — 대상이 TestClient 응답으로 바뀐 것뿐
 - 응답 객체 형태는 거의 동일 — status_code, headers, body 동일 접근

4. 응답을 보기 좋게 출력하는 함수 만들기

여러 요청 결과를 반복해서 확인할 것이므로, 응답 내용을 보기 좋게 출력하는 함수를 만들어둡니다.

```
def print_api_response(response):  
    print("=" * 60)  
    print("Status Code:", response.status_code)  
    print("-" * 60)  
  
    try:  
        print("Response Body:")  
        print(json.dumps(response.json(), ensure_ascii=False, indent=2))  
    except Exception:  
        print("Response Text:")  
        print(response.text)  
  
    print("=" * 60)
```

Python



5. /health 요청 테스트 하기

- 입력값이 필요 없는 가장 단순한 Endpoint 테스트
- 실습 포인트:
 - client.get('/health') 한 줄로 200 응답이 돌아오는지 확인
 - TestClient가 우리 앱에 정상 연결됐는지 첫 검증

5. /health 요청 테스트하기

먼저 입력값이 필요 없는 가장 단순한 API를 테스트합니다.

```
response = client.get("/health")  
  
print_api_response(response)
```

Python



6. Path Parameter 테스트하기

- URL 경로 안의 값을 함수 인자로 받는 Endpoint 테스트
- 실습 포인트:
 - /items/3은 정상 처리, /items/abc는 검증 실패 — 타입 힌트 int가 자동 검증
 - if 문 없이 함수 인자에 타입만 적었는데 FastAPI가 알아서 차단
 - 실패 응답의 Status Code도 함께 확인

6. Path Parameter 테스트하기

/items/{item_id}는 URL 경로 안의 값을 함수 인자로 받는 Endpoint입니다.

먼저 정상 요청을 보내봅니다.

```
response = client.get("/items/3")  
print_api_response(response)
```

Python

이번에는 숫자가 아닌 값을 넣어 실패 응답을 확인합니다.

```
response = client.get("/items/abc")  
print_api_response(response)
```

Python



7. Query Parameter 테스트하기

- Query Parameter가 함수 인자로 전달되는지 확인
- 실습 포인트:
 - /search에 keyword·limit 붙여 호출 — params=dict로 자동 인코딩
 - Query Parameter 생략 시 기본값이 사용되는지 함께 확인

7. Query Parameter 테스트하기

/search 는 Query Parameter를 통해 검색 조건을 받는 Endpoint입니다.

```
response = client.get("/search", params={
    "keyword": "fastapi",
    "limit": 5
})

print_api_response(response)
```

Python

Query Parameter를 생략하면 기본값이 사용되는지 확인합니다.

```
response = client.get("/search")

print_api_response(response)
```

Python



8. Request Body 테스트하기

- POST 요청에서 JSON Body가 Pydantic 모델로 전달되는지 확인
- 실습 포인트:
 - 정상 요청은 message 그대로 반환, 빈 Body는 함수 실행 전 차단
 - 차단 시 Status Code 422 — **Pydantic의 자동 검증** (이번 실습 하이라이트)
 - message에 문자열 대신 숫자/리스트를 넣어 응답 차이 확인 권유

8. Request Body 테스트하기

POST /message 는 JSON Body를 입력으로 받는 Endpoint입니다.

먼저 정상 요청을 보내봅니다.

```
response = client.post("/message", json={
    "message": "hello fastapi"
})

print_api_response(response)
```

Python

이번에는 필수 필드인 message 를 빼고 요청해봅니다.

```
response = client.post("/message", json={})

print_api_response(response)
```

Python



9. 자동 생성된 API 문서 정보 확인하기

- FastAPI가 생성한 OpenAPI 정보를 코드로 확인
- 실습 포인트:
 - `app.openapi()` 호출 — FastAPI가 자동 생성한 문서 정보를 dict로 받음
 - `paths`의 key를 출력해 등록된 모든 Endpoint가 잘 들어있는지 확인

9. 자동 생성된 API 문서 정보 확인하기

FastAPI는 작성한 Endpoint 정보를 바탕으로 API 문서를 자동 생성합니다.

브라우저에서는 다음 주소로 FastAPI 자동 문서 화면을 확인할 수 있습니다.

`http://127.0.0.1:8000/docs`

노트북에서는 `app.openapi()` 를 통해 API 문서의 기반이 되는 OpenAPI 정보를 확인할 수 있습니다.

```
openapi_schema = app.openapi()

print("API 제목:", openapi_schema["info"]["title"])
print("등록된 Path 목록:")

for path in openapi_schema["paths"].keys():
    print("-", path)
```

Python



9. 자동 생성된 API 문서 정보 확인하기

- Path와 Method 정보 확인
- 예상 출력:
 - /health: GET
 - /items/{item_id}: GET
 - /search: GET
 - /message: POST
- 실습 포인트:
 - /health → GET, /items/{item_id} → GET, /search → GET, /message → POST
 - 코드로 만든 API 구조가 그대로 문서 정보에 반영 — /docs도 이 정보 기반

각 Path에 어떤 Method가 등록되어 있는지도 확인합니다.

```
for path, methods in openapi_schema["paths"].items():  
    method_names = ", ".join(methods.keys()).upper()  
    print(f"{path}: {method_names}")
```

Python



10. 서버 파일로 저장하고 /docs 확인하기

- 노트북에서 만든 앱을 main.py로 저장

10. 서버 파일로 저장하고 /docs 확인하기

아래 셀을 실행하면 지금 만든 FastAPI 앱을 main.py 파일로 저장합니다.

```
%%writefile main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

@app.get("/health")
def health_check():
    return {
        "status": "ok"
    }

@app.get("/items/{item_id}")
def get_item(item_id: int):
    return {
        "item_id": item_id
    }

@app.get("/search")
def search_items(keyword: str = "all", limit: int = 10):
    return {
        "keyword": keyword,
        "limit": limit
    }

class MessageRequest(BaseModel):
    message: str

@app.post("/message")
def create_message(request: MessageRequest):
    return {
        "message": request.message
    }
```



10. 서버 파일로 저장하고 /docs 확인하기

- 터미널 실행 및 브라우저에서 문서 확인

터미널에서 서버를 실행합니다.

```
uvicorn main:app --reload
```

브라우저에서 다음 주소로 접속해 FastAPI 자동 문서 화면을 확인합니다.

```
http://127.0.0.1:8000/docs
```

- 실습 포인트:
 - uvicorn main:app --reload로 서버 띄우고 /docs 접속 — Swagger UI 자동 생성
 - 노트북에서 본 모든 Endpoint가 자동 문서에 표시 — [코드가 곧 문서](#)
 - Try it out으로 브라우저에서 직접 호출 — TestClient와 결과 비교



TODO

- TODO 1. /search 요청 테스트하기
- TODO 2. /items/{item_id} 실패 요청 테스트하기
- TODO 3. /message 정상 요청과 실패 요청 비교하기
- TODO 4. OpenAPI Path 목록 확인하기



실습 정리

- TestClient를 사용해 실습 환경 안에서 API 요청을 실행
- /docs 화면에서 FastAPI 자동 문서를 확인
- Path Parameter, Query Parameter, Request Body 각각 테스트
- 잘못된 입력은 Endpoint 처리 전에 검증 실패 응답으로 반환
- app.openapi()를 통해 자동 문서의 기반 정보를 코드로 확인

텍스트 처리 API 구현 (실습)



실습 목표

- FastAPI와 Pydantic 모델을 활용해 입력 텍스트를 처리하는 API를 구현하고,
요청 모델·응답 모델·처리 로직·Endpoint 연결 흐름을 실습한다

실습 2. 텍스트 처리 API 구현

실습 목표

이번 실습은 FastAPI와 Pydantic 모델을 사용해 텍스트 처리 API(`POST /text/process`)를 처음부터 끝까지 만들어 보는 단계입니다. 실제 LLM은 호출하지 않고 규칙 기반 처리 로직으로 API 흐름을 완성하며, 2일차 이론에서 다룬 Request Model · Response Model · 자동 검증이 한 자리에 모이는 자리입니다.

다음은 차례로 만들어 봅니다.

- Request Model(`text` · `mode`)과 Response Model(`result`) 정의
- 처리 로직을 별도 함수(`process_text`)로 분리해 작성·검증
- `@app.post` Endpoint에서 Request → 처리 함수 → Response Model 순으로 연결
- TestClient로 mode별 정상 응답 확인 (`upper` · `lower` · `reverse` · `length`)
- `mode` 기본값 동작과 잘못된 입력의 검증 흐름 확인



실습 흐름

1 라이브러리 불러오기

2 구현할 API 구조 살펴보기

3 FastAPI 앱 만들기

4 Request Model 정의하기

5 Response Model 정의하기

6 텍스트 처리 함수 만들기

7 처리 함수 실행해보기

8 Endpoint 구현하기

9 TestClient로 API 요청 보내기

10 응답 출력 함수 만들기

11 기본 mode 요청 테스트하기

12 mode 기본값 테스트하기

13 서버 파일로 저장하기

14 자동 문서 화면에서 테스트하기

TODO

process_text_v2 · ResponseV2 모델 · /text/process-v2 등 v2 API를 직접 구현



1. 라이브러리 불러오기

- FastAPI 앱 생성, 요청 테스트, 데이터 모델 정의에 필요한 라이브러리를 불러옵니다.

1. 라이브러리 불러오기

FastAPI 앱 생성, 요청 테스트, 데이터 모델 정의에 필요한 라이브러리를 불러옵니다.

```
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel
import json
```

Python



2. 구현할 API 구조 살펴보기

- 입력 예시
 - POST /text/process
 - {"text": "Hello", "mode": "upper"}
 - mode 값: upper · lower · reverse · length
- 응답 예시
 - {"result": "HELLO"}

2. 구현할 API 구조 살펴보기

이번 실습에서 만들 기본 API는 입력 텍스트와 처리 방식을 받아 결과를 반환합니다.

POST /text/process

지원할 기본 mode 는 다음과 같습니다.

upper	→ 대문자로 변환
lower	→ 소문자로 변환
reverse	→ 문자열 뒤집기
length	→ 글자 수 반환



3. FastAPI 앱 만들기

- 실습 포인트:
 - 텍스트 처리 API를 등록할 FastAPI 앱 객체 생성
 - FastAPI() 괄호 안에 **title 지정** — /docs 화면 상단에 API 이름으로 표시
 - 이후 모든 Endpoint는 이 app 객체에 등록

3. FastAPI 앱 만들기

텍스트 처리 API를 등록할 FastAPI 앱 객체를 생성합니다.

```
app = FastAPI(title="Text Processing API")
```

Python



4. Request Model 정의하기

- Request Model은 클라이언트가 보내는 입력 Body의 구조를 정의함
- 실습 포인트:
 - text는 필수 입력, mode는 기본값 'upper'가 있는 선택 입력
 - 한 모델에 두 필드 — 함수 인자에 두 자리가 생긴 셈
 - 잠시 후 mode를 빠뜨린 요청 보내 기본값이 동작하는지 확인

4. Request Model 정의하기

Request Model은 클라이언트가 보내는 입력 Body의 구조를 정의합니다.

이번 API는 `text` 와 `mode` 를 입력으로 받습니다.

```
class TextProcessRequest(BaseModel):  
    text: str  
    mode: str = "upper"
```

Python



5. Response Model 정의하기

- Response Model은 서버가 반환하는 응답 Body의 구조를 정의함
- 실습 포인트:
 - result 필드 하나로 처리 결과 담을 자리 만들기
 - Request-Response가 같은 BaseModel 상속 — 모양은 같고 **방향만 반대**
 - response_model 옵션에 이 클래스 연결 → 응답 모양 자동 약속

5. Response Model 정의하기

Response Model은 서버가 반환하는 응답 Body의 구조를 정의합니다.

기본 API는 처리 결과를 `result` 필드로 반환합니다.

```
class TextProcessResponse(BaseModel):  
    result: str
```

Python



6. 텍스트 처리 함수 만들기

- Endpoint 안에 모든 로직을 직접 쓰지 않고, 처리 함수를 따로 만듦
- 실습 포인트:
 - Endpoint는 HTTP 입출구만, 처리 로직은 process_text가 담당
 - 역할이 나뉘면 처리 함수만 따로 테스트하기도 쉬워짐
 - 5일차의 Service 분리로 그대로 이어지는 사고

Endpoint 안에 모든 로직을 직접 작성하지 않고, 텍스트 처리 함수를 따로 만듭니다.

이렇게 하면 요청을 받는 부분과 실제 처리 로직을 분리해서 볼 수 있습니다.

```
def process_text(text: str, mode: str) -> str:
    if mode == "upper":
        return text.upper()

    if mode == "lower":
        return text.lower()

    if mode == "reverse":
        return text[::-1]

    if mode == "length":
        return str(len(text))

    return text
```



7. 처리 함수 실행해보기

- Endpoint에 연결하기 전에 함수만 먼저 실행
- 실습 포인트:
 - API 서버 없이 처리 로직만 먼저 검증 — 단위 테스트의 첫 형태
 - mode를 upper/lower/reverse로 바꿔가며 결과 비교
 - 내부 함수가 안정적이면 Endpoint도 안정적

7. 처리 함수 실행해보기

서버 Endpoint에 연결하기 전에, 처리 함수가 원하는 결과를 반환하는지 먼저 실행해봅니다.

```
sample_text = "Hello FastAPI"

for mode in ["upper", "lower", "reverse", "length"]:
    result = process_text(sample_text, mode)

    print("=" * 50)
    print("mode:", mode)
    print("result:", result)
```

Python



8. Endpoint 구현하기

- Request Model로 입력을 받고, 처리 함수를 호출한 뒤, Response Model 형태로 응답
- 실습 포인트:
 - 5줄 안에 검증·처리·응답 정리 모두 등장 — Endpoint는 다리만 놓음
 - `response_model=TextResponse` 한 줄이 응답 모양을 자동 검증·고정

8. Endpoint 구현하기

Request Model로 입력을 받고, 텍스트 처리 함수를 호출한 뒤, Response Model 형태로 응답합니다.

```
@app.post("/text/process", response_model=TextProcessResponse)
def process_text_api(request: TextProcessRequest):
    result = process_text(request.text, request.mode)

    return {
        "result": result
    }
```

Python



9. TestClient로 API 요청 보내기

- 실습 포인트:
 - 별도 브라우저 없이 노트북 안에서 API 요청 실행
 - 정상 요청과 실패 요청을 코드로 바로 비교 가능
 - 앞 실습과 동일한 패턴 — TestClient 사용법은 한 번 익히면 끝

9. TestClient로 API 요청 보내기

TestClient 를 사용해 노트북 안에서 API 요청을 실행합니다.

```
client = TestClient(app)
```

Python



10. 응답 출력 함수 만들기

- 실습 포인트:
 - 앞 실습의 print_api_response 함수와 동일 — 두 실습이 독립 실행 가능하도록 재정의
 - 응답을 같은 형식으로 정리해 비교하기 쉽게 만드는 패턴

10. 응답 출력 함수 만들기

요청 결과를 보기 좋게 출력하는 함수를 만듭니다.

(실습1에서 만든 print_api_response 함수와 동일합니다. 실습1·2가 각각 독립적으로 실행될 수 있도록 다시 정의합니다.)

```
def print_api_response(response):  
    print("=" * 60)  
    print("Status Code:", response.status_code)  
    print("-" * 60)  
  
    try:  
        print("Response Body:")  
        print(json.dumps(response.json(), ensure_ascii=False, indent=2))  
    except Exception:  
        print("Response Text:")  
        print(response.text)  
  
    print("=" * 60)
```

Python



11. 기본 mode 요청 테스트하기

- 실습 포인트:
 - 같은 Endpoint에 **mode 값만 바꿔 호출** — 같은 입력에서 결과가 어떻게 달라지는지
 - Request Body → 처리 로직 → Response Body 전체 파이프라인이 한 번에 동작
 - 지원하지 않는 mode를 일부러 보내 처리 함수의 else 가지 동작 확인

11. 기본 mode 요청 테스트하기

upper, lower, reverse, length 요청을 차례대로 보내봅니다.

```
test_requests = [  
    {"text": "Hello FastAPI", "mode": "upper"},  
    {"text": "Hello FastAPI", "mode": "lower"},  
    {"text": "Hello FastAPI", "mode": "reverse"},  
    {"text": "Hello FastAPI", "mode": "length"},  
]  
  
for request_body in test_requests:  
    print("요청 Body:", request_body)  
    response = client.post("/text/process", json=request_body)  
    print_api_response(response)
```

Python



12. mode 기본값 테스트하기

- mode 생략 시 Request Model의 기본값이 사용됨
- 실습 포인트:
 - Body에 text만 보내도 mode='upper'로 처리 — 모델 기본값의 실제 효과
 - Pydantic 기본값이 코드 한 줄로 자동 적용되는 모습

12. mode 기본값 테스트하기

`mode` 를 생략하면 Request Model에 정의한 기본값 `"upper"` 가 사용됩니다.

```
response = client.post("/text/process", json={
    "text": "default mode test"
})

print_api_response(response)
```

Python



13. 서버 파일로 저장하기

- 노트북에서 만든 API 코드를 main.py로 저장
- 실습 포인트:
 - 노트북 셀이 의도치 않게 다른 순서로 실행되어도 결과 흔들리지 않게
 - 이후 uvicorn main:app --reload로 실제 서버 형태로 실행 가능

13. 서버 파일로 저장하기

아래 셀을 실행하면 지금까지 만든 API 코드를 main.py 파일로 저장합니다.

```
%%writefile main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Text Processing API")

class TextProcessRequest(BaseModel):
    text: str
    mode: str = "upper"

class TextProcessResponse(BaseModel):
    result: str

def process_text(text: str, mode: str) -> str:
    if mode == "upper":
        return text.upper()

    if mode == "lower":
        return text.lower()

    if mode == "reverse":
        return text[::-1]

    if mode == "length":
        return str(len(text))

    return text

@app.post("/text/process", response_model=TextProcessResponse)
def process_text_api(request: TextProcessRequest):
    result = process_text(request.text, request.mode)

    return {
        "result": result
    }
```



14. FastAPI 자동 문서 화면에서 테스트하기

- 터미널 실행 및 브라우저에서 문서 확인
- 실습 포인트:
 - POST /text/process가 자동 문서 화면에 표시 — [코드가 곧 문서](#)
 - Request Body 입력창에서 text, mode를 직접 넣어 테스트
 - TestClient 결과와 브라우저 결과가 동일한지 비교

14. FastAPI 자동 문서 화면에서 테스트하기

터미널에서 서버를 실행합니다.

```
uvicorn main:app --reload
```

브라우저에서 다음 주소로 접속합니다.

```
http://127.0.0.1:8000/docs
```

POST /text/process 를 열고 다양한 mode 값을 입력해 실행해봅니다.



TODO 준비

- HTTPException으로 에러 응답 반환
 - 지금까지는 Pydantic 검증 실패가 자동으로 422 응답으로 처리됨
 - 직접 에러 응답을 만들고 싶을 때는 HTTPException 사용
- 핵심:
 - 자동 검증(422)은 Pydantic이 처리
 - 비즈니스 로직 에러(400 등)는 HTTPException으로 직접 발생

main.py

```
from fastapi import HTTPException

if 조건이 잘못된 경우:
    raise HTTPException(
        status_code=400,
        detail="잘못된 요청 메시지"
    )
```

422 자동 검증

Pydantic이 검증 실패를 자동 처리

직접 에러 처리

400 404 500

HTTPException으로 직접 발생



TODO

- TODO 1. process_text_v2() 함수 구현하기
- TODO 2. TextProcessResponseV2 모델 정의하기
- TODO 3. /text/process-v2 Endpoint 구현하기
- TODO 4. TestClient로 v2 API 테스트하기
- TODO 5. 최종 서버 파일 저장하기



실습 정리

- Request Model로 입력 Body 구조를 정의
- Response Model로 응답 Body 구조를 정의함
- 텍스트 처리 로직을 별도 함수로 분리함
- Endpoint에서 Request Model을 받아 처리 함수를 호출함
- TestClient와 FastAPI 자동 문서 화면으로 API 요청을 테스트
- v2 API에서 응답 구조 확장과 오류 처리를 구현함